

Toric Algebraic Training Objectives for ToricGT

Combinatorial Commutative Algebra, Reasoning Trajectories, and Bits-per-byte Compression

Amelie Schreiber

June 2026

Abstract

This paper develops a toric-algebraic training interface for ToricGT-style reasoning models. It reviews Part II, “Toric Algebra”, of Miller–Sturmfels *Combinatorial Commutative Algebra* and turns its core objects—semigroup rings, lattice ideals, Hilbert bases, initial ideals, multigraded Betti numbers, K-polynomials, multidegrees, Laurent monomial modules, hull and Scarf resolutions, toric quotients, irreducible and injective resolutions, Ehrhart/Brion generating functions, and toric local cohomology—into concrete objectives for neural reasoning. The guiding modeling decision is to treat a reasoning trajectory as a finite, quantized, multigraded object in an embedding space: a graph-of-thought path or directed acyclic graph produces step increments, active ReLU/tropical cells, retrieval keys, proof obligations, verifier states, memory references, and behavior coordinates. These data generate semigroups, congruences, lattice ideals, modules, and finite cell complexes. The algebraic invariants of those objects become training losses, audit metrics, regularizers, curricula, and retrieval certificates.

The emphasis is pragmatic. Every proposed auxiliary is subordinated to held-out bits-per-byte, accuracy, proof validity, retrieval quality, and behavior-control metrics. The paper gives formal definitions, proof sketches, and implementation recipes for more than twenty new objectives, of which at least ten are directly pulled from Part II of the book. Several results are elementary but useful: binomial relation losses are exactly the obstruction to factoring a predicted trajectory code through a semigroup ring; initial-ideal training gives conservative homological upper bounds; saturation-hole penalties detect nonnormal reasoning semigroups; Scarf genericity makes a lattice-resolution certificate minimal; Brion-style vertex decompositions yield sparse retrieval caches; and local-cohomology cocomplexes provide a computable coverage diagnostic for missing reasoning regions. The intended use is not to make a deployed language model carry a symbolic algebra system. The intended use is to use small, cached, auditable commutative-algebra certificates during training, and to export only the compact pieces whose ablations improve held-out bits-per-byte or predeclared reasoning behavior.

Contents

1	Motivation and relation to the ToricGT series	4
1.1	What this paper claims and what it does not claim	4
1.2	High-level pipeline	5
2	Part II of <i>Combinatorial Commutative Algebra</i> as a training manual	5
2.1	Chapter 7: semigroup rings, lattice ideals, Hilbert bases, and initial ideals	5
2.2	Chapter 8: multigradings, K-polynomials, Betti numbers, and multidegrees	5
2.3	Chapter 9: syzygies of lattice ideals	6
2.4	Chapter 10: toric varieties as multigraded quotients	6
2.5	Chapter 11: irreducible and injective resolutions	6
2.6	Chapter 12: Ehrhart polynomials, dualizing complexes, Brion’s formula, and short rational functions	6

2.7	Chapter 13: local cohomology	7
2.8	A map from Part II to ToricGT components	7
3	Reasoning trajectories as semigroup, lattice, and module data	7
3.1	Quantized reasoning-step alphabets	7
3.2	The bits-per-byte gate	8
3.3	Reasoning modules and filtered complexes	9
4	Toric algebra and ReLU/tropical neural computation	9
4.1	Active regions as weight orders	9
4.2	Why initial degenerations help training	10
4.3	ReLU network applications	10
5	Catalog of toric-algebraic applications, losses, metrics, techniques, and regularizers	11
5.1	O1. Semigroup trajectory congruence loss	11
5.2	O2. Saturation-hole normality regularizer	11
5.3	O3. Hilbert-basis step dictionary pruning	12
5.4	O4. Groebner-shadow curriculum	12
5.5	O5. Multigraded K-polynomial trajectory signature	12
5.6	O6. Betti-table reasoning regularity	13
5.7	O7. Multidegree class matching	13
5.8	O8. Initial-Betti upper-semicontinuity gate	13
5.9	O9. Laurent periodic trajectory module	14
5.10	O10. Hull-resolution trajectory scaffold	14
5.11	O11. Scarf-generic minimal certificate	14
5.12	O12. Lawrence-lift squarefree augmentation	15
5.13	O13. Toric quotient invariance head	15
5.14	O14. Cox-style semistable routing window	15
5.15	O15. Irreducible-component hallucination barrier	16
5.16	O16. Bass-number failure-mode classifier	16
5.17	O17. Dualizing-complex interior evidence prior	16
5.18	O18. Ehrhart context-growth law	17
5.19	O19. Brion vertex-cone retrieval cache	17
5.20	O20. Short rational generating-function code	17
5.21	O21. Cech-support local-cohomology loss	18
5.22	O22. Toric local cohomology coverage audit	18
5.23	O23. Cohen-Macaulay reasoning-depth objective	18
5.24	O24. Universal grading discovery	19
5.25	O25. Markov-basis negative sampling	19
6	Training paradigms derived from Part II	19
6.1	Paradigm A: audit-only algebraic cartography	19
6.2	Paradigm B: monomial-shadow pretraining and binomial lifting	20
6.3	Paradigm C: ReLU/tropical chart regularization	20
6.4	Paradigm D: resolution-shaped graph-of-thought training	20
6.5	Paradigm E: quotient and behavior control	20
6.6	Paradigm F: retrieval and memory as semigroup modules	21

7	Implementation methodology	21
7.1	Data schema	21
7.2	Certificate generators	21
7.3	GPU integration	22
7.4	Quantization and artifact accounting	22
7.5	Reproducible evaluation	22
8	Worked examples	23
8.1	A commuting-proof semigroup	23
8.2	A binomial relation for equivalent decomposition	23
8.3	A saturation hole	24
8.4	A Scarf certificate for generic relation data	24
8.5	Local cohomology as evidence-hole detection	24
9	Additional theoretical guarantees	24
10	A concrete experimental plan	25
10.1	Synthetic algebra suite	25
10.2	Reasoning suite	25
10.3	Ablations	26
10.4	Stopping rules	26
10.5	Expected failure modes	26
11	Expanded theory: why toric algebra can change learning, not only diagnostics	27
11.1	A finite toric algebra certificate functor	27
11.2	Minimum-description interpretation of Hilbert bases	28
11.3	Initial ideals as ReLU and tropical shadows	28
11.4	K-polynomials as compact multi-step signatures	29
11.5	Local cohomology as coverage and uncertainty	30
12	Detailed Part-II-derived objective cards	30
13	Reproducibility appendix: data, code paths, and exact audits	32
13.1	Canonical record format	32
13.2	Exact certificate generation	33
13.3	Deterministic validation and leakage control	33
13.4	Recommended hyperparameters	34
13.5	Minimal open-source reproduction	34
13.6	Reporting checklist	34
14	Wild-card research directions	35
14.1	Toric algebra for mixture-of-experts routing	35
14.2	Semigroup-valued curriculum scheduling	35
14.3	Universal grading discovery for hidden features	35
14.4	Canonical-module evidence priors	35
14.5	Multigraded Hilbert-scheme view of representation drift	36
14.6	Integer-programming certificates as tool-use supervision	36
15	Conclusion	36

1 Motivation and relation to the ToricGT series

ToricGT began as a graph-token model for structured reasoning. The architectural core is simple: tokenize nodes, edges, and graph-level records; process the resulting table with permutation-equivariant Transformer blocks; use tropical, toric, noncommutative-phase, GraphCG, GFlowNet, retrieval, and low-rank certificate modules only when the task supplies the relevant structure. The original ToricGT paper made the toric component operational by attaching finite BGG/Koszul/Tate/Euler–Koszul certificates to hidden trajectories. ToricGT II then moved from static certificates to iterative reasoning dynamics: a reasoning step carried a filtered simplicial complex, a multigraded persistence module, sheaf/vector-bundle features, and toric chart codes, and repeated application of the model was treated as a dynamical system. ToricGT III externalized memory through model-context-protocol traces, markdown memories, and knowledge graphs vectorized with adjacency and simplicial information. ToricGT IV proposed oracle trajectory classifiers for accuracy, tone, personality corridor, compliance, retrieval hygiene, and efficiency.

The present paper is narrower and more algebraic. Its source is Part II, “Toric Algebra”, of Miller and Sturmfels. That part is not merely background for toric varieties. It is a manual for turning combinatorics into exact algebra: finite semigroups become rings, congruence relations become lattice ideals, Hilbert bases are minimal dictionaries, initial ideals are degenerations, multigraded Betti numbers quantify syzygies, Laurent monomial modules make lattice ideals look like infinite periodic monomial ideals, toric quotients encode invariance, injective and irreducible resolutions expose components and failure modes, Ehrhart/Brion theory compresses lattice counts, and local cohomology computes missing support through polyhedral cocomplexes. These objects match the needs of a training system that wants compact, auditable, symbolic shadows of hidden reasoning without paying for a symbolic engine at inference time.

We use the following design principle throughout.

A toric-algebraic auxiliary is useful for ToricGT only if it gives a finite certificate, a differentiable surrogate, and a score-first gate. The exact certificate must be reproducible from the training record; the surrogate must be small enough to run on GPU; and the component is retained only when ablations improve held-out bits-per-byte, reasoning quality, retrieval quality, or behavior control without violating artifact budgets.

This principle prevents a common failure mode in mathematically decorated machine learning: beautiful invariants are added because they are beautiful. Here beauty is secondary. The algebra is valuable because it gives reusable coordinates for compression and reasoning. For instance, a lattice ideal generated by binomials can encode when two superficially different proof paths are equivalent; a multigraded Betti table can detect whether the model uses a short resolution-like argument or a bloated set of redundant obligations; a local-cohomology cocomplex can identify missing evidence chambers; and an Ehrhart generating function can summarize how a family of admissible reasoning states grows as context length increases.

1.1 What this paper claims and what it does not claim

The theory here has two levels. The first level consists of exact finite claims. If a training record supplies a semigroup, lattice, module, finite complex, or rational polyhedral object, the associated algebraic operations are exact and reproducible. Theorems about binomial factorization, initial degenerations, Scarf minimality under genericity, injective-resolution component detection, Brion-style decompositions, and local-cohomology cocomplexes apply to those finite objects. The second level consists of empirical modeling hypotheses: that forcing hidden trajectories to respect some of these finite objects can improve held-out bits-per-byte, reasoning accuracy, and behavior control. Those hypotheses are not asserted as theorems. They are experimental claims to be tested by matched ablations, exported-artifact evaluation, and hard held-out tasks.

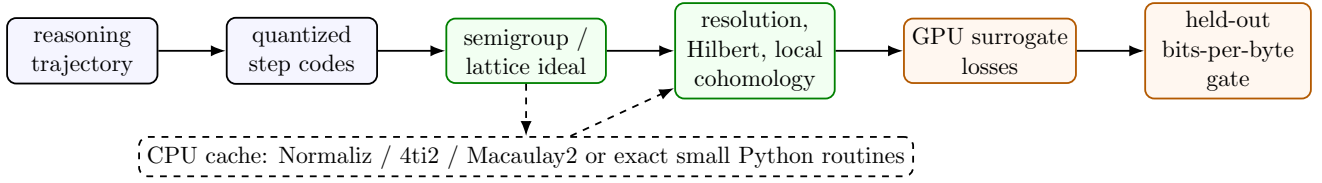


Figure 1: The toric-algebraic training interface. A hidden reasoning trajectory is quantized into a finite algebraic object. Exact certificates are computed offline or cached. Small GPU losses train the model. Deployment keeps only components that pass held-out bits-per-byte and reasoning gates.

1.2 High-level pipeline

2 Part II of *Combinatorial Commutative Algebra* as a training manual

Part II, “Toric Algebra”, has seven chapters. The following review is intentionally selective: each topic is read through the lens of ToricGT training.

2.1 Chapter 7: semigroup rings, lattice ideals, Hilbert bases, and initial ideals

A semigroup ring $\mathbb{k}[Q]$ is the algebra with basis $\{t^a : a \in Q\}$ and multiplication $t^a t^b = t^{a+b}$. If Q is generated by $a_1, \dots, a_n$, then the polynomial ring $S = \mathbb{k}[x_1, \dots, x_n]$ maps onto $\mathbb{k}[Q]$ by $x_i \mapsto t^{a_i}$. The kernel is the lattice ideal

$$I_L = \langle x^u - x^v : u, v \in \mathbb{N}^n, u - v \in L \rangle,$$

where $L \subseteq \mathbb{Z}^n$ is the lattice of relations among the generators. This is the first direct bridge to reasoning models. If a reasoning trajectory can be described by counts $u \in \mathbb{N}^n$ of primitive moves, then a lattice relation $u - v \in L$ says that two move multisets produce the same abstract reasoning state. A binomial residual $x^u - x^v$ is therefore an equivalence residual.

The book emphasizes that affine semigroup structure is polyhedral. A pointed affine semigroup has a unique finite minimal generating set. For a rational pointed cone C , Gordan’s lemma gives finite generation of $C \cap A$; its unique minimal generating set is the Hilbert basis. In machine-learning language, a Hilbert basis is a minimal reusable dictionary for lattice-respecting states. If the learned step dictionary is larger than the Hilbert basis, the model may be wasting parameters; if it is smaller, it may be missing states and forcing brittle interpolation.

Initial ideals of lattice ideals under weight orders are the algebraic form of degenerating a binomial problem to a monomial one. For ToricGT this is exactly what a ReLU or tropical head often does: it chooses an active monomial, active predecessor, active face, or active proof obligation by a weight comparison. Thus initial ideals provide a rigorous language for training curricula that begin with monomial shadows and lift to binomial consistency.

2.2 Chapter 8: multigradings, K-polynomials, Betti numbers, and multidegrees

A multigrading by an abelian group A assigns $\deg(x_i) = a_i$. Hilbert series and K-polynomials become multigraded summaries of which states exist at each degree. In ToricGT, a degree can encode token budget, proof depth, tool permissions, memory source, verifier confidence, behavior corridor, algebraic task type, or GraphCG coordinate bucket. The K-polynomial is the alternating Euler characteristic of a free resolution; it summarizes a complex without storing every chain group.

A central training lesson is upper semicontinuity: passing to an initial submodule preserves the Hilbert series but can only increase Betti numbers. This gives a conservative diagnostic. If the monomial

or ReLU-degenerated shadow of a trajectory has large Betti numbers, then the true hidden algebraic object cannot have larger efficiency than the shadow suggests. The shadow can be used as an inexpensive upper-bound monitor for syzygetic complexity.

Multidegrees are equivariant Chow-class-like invariants. In ToricGT they can serve as coarse but stable fingerprints of reasoning families. They are less brittle than exact Betti tables and better aligned with geometric generalization across equivalent embeddings.

2.3 Chapter 9: syzygies of lattice ideals

Chapter 9 explains why lattice ideals behave like infinite periodic monomial ideals. Laurent monomial modules $M \subseteq \mathbb{k}[x_1^{\pm 1}, \dots, x_n^{\pm 1}]$ replace finite monomial staircases by periodic or infinite staircases. Hull complexes and Scarf complexes then give cellular resolutions. For a generic Laurent monomial module, the Scarf complex, hull resolution, and minimal free resolution coincide. For a generic lattice ideal, the minimal free resolution is the image of the Scarf complex.

For ToricGT this is a blueprint for trajectory-memory compression. Many reasoning trajectories differ by periodic rewrites: commute independent steps, swap equivalent evidence orderings, rotate through a template, or translate along a memory loop. A lattice-module representation lets the model learn the periodic quotient instead of memorizing every translate.

2.4 Chapter 10: toric varieties as multigraded quotients

Chapter 10 explains how multigraded polynomial rings produce toric varieties through abelian group actions, projective quotients, and Cox-like constructions. For training, the important point is quotienting. Many nuisance transformations in reasoning should be quotiented out: graph relabeling, synonym choice, tool-call identifier choice, equivalent proof-step order, and benign style variation. A toric quotient objective trains the hidden representation to preserve the quotient-invariant part while retaining enough local data for decoding.

2.5 Chapter 11: irreducible and injective resolutions

Free resolutions describe generators and syzygies. Injective and irreducible resolutions describe components, supports, and failure modes. For a model, these are not interchangeable. A free-resolution objective asks, “Are the proof obligations generated efficiently?” An injective/irreducible objective asks, “Where can the reasoning fail, and what components are responsible?” Bass numbers are the dual homological invariants to Betti numbers. In a verifier or hallucination-control setting, Bass-like features are natural classifiers of missing support and irreducible failure components.

2.6 Chapter 12: Ehrhart polynomials, dualizing complexes, Brion’s formula, and short rational functions

Ehrhart theory counts lattice points in dilates of a polytope. It therefore counts how many discrete states are available at a budget scale. In ToricGT, dilating a polytope can mean increasing reasoning length, evidence budget, context window, number of retrieved memories, or number of allowed graph-of-thought branches. Brion’s formula decomposes a lattice-point generating function into vertex contributions; this is a natural retrieval cache design: instead of storing all states, store vertex-cone summaries and reconstruct counts or priors locally. Short rational generating functions give compact count surrogates useful for bits-per-byte and artifact budgets.

2.7 Chapter 13: local cohomology

Local cohomology measures sections supported on a closed set. It can be computed with Čech complexes; for semigroup rings, toric local cohomology reduces degree-wise to cohomology of polyhedral cocomplexes. This is directly useful for auditing missing reasoning regions. A model may perform well in ordinary likelihood while having holes: missing evidence combinations, unsupported behavior corridors, or disconnected retrieval regions. Local cohomology provides a way to detect these holes as homological obstructions rather than as scattered errors.

2.8 A map from Part II to ToricGT components

Book object	Training interpretation	Example ToricGT use
Semigroup Q	allowed finite compositions of primitive reasoning moves	trajectory state dictionary, retrieval-key algebra
Lattice ideal I_L	equivalence of move multisets	binomial relation loss; proof-path equivalence
Hilbert basis	minimal nonredundant generator dictionary	step-code pruning; low-byte basis selection
Initial ideal	ReLU/tropical degeneration	monomial shadow curriculum; active-cell audit
K-polynomial	compressed Euler characteristic of a resolution	low-byte signature for trajectory family
Multigraded Betti table	distribution of generators and syzygies by degree	reasoning-depth regularity; proof-obligation complexity
Scarf complex	minimal generic cellular resolution	sparse certificate for generic trajectory lattice
Toric quotient	invariant representation under group action	graph relabeling, style, and tool-ID invariance
Injective resolution	component/failure support	hallucination and missing-evidence classifiers
Ehrhart series	growth of admissible states with budget	context-length planning and retrieval capacity
Brion decomposition	vertex-local generating functions	sparse vector database/retrieval cache
Local cohomology	missing support / holes	coverage loss for evidence, tools, and behavior corridors

3 Reasoning trajectories as semigroup, lattice, and module data

3.1 Quantized reasoning-step alphabets

Let G_t be the graph-of-thought state at step t , including hidden state h_t , active cells, retrieved memories, verifier fields, and behavior coordinates. A quantizer

$$q_\phi(G_t, G_{t+1}) \in \{1, \dots, n\}$$

assigns a primitive move type to the transition. A completed trajectory τ then gives a count vector $u(\tau) \in \mathbb{N}^n$, where $u_i(\tau)$ is the number of times primitive move i was used. A step dictionary assigns

each primitive move a degree $a_i \in A$, where A is an abelian group such as $\mathbb{Z}^d \oplus \mathbb{Z}/c_1 \oplus \dots$. The degree of a trajectory is

$$\deg(\tau) = \sum_{i=1}^n u_i(\tau) a_i.$$

The semigroup $Q = \mathbb{N}\{a_1, \dots, a_n\} \subseteq A$ is the set of reachable abstract degrees.

Definition 3.1 (Toric trajectory presentation). A toric trajectory presentation is a tuple

$$\mathcal{P} = (A, a_1, \dots, a_n, L, S, I_L),$$

where A is an abelian grading group, $a_i \in A$, $L = \ker(\mathbb{Z}^n \rightarrow A)$, $S = \mathbb{k}[x_1, \dots, x_n]$ with $\deg(x_i) = a_i$, and $I_L = \langle x^u - x^v : u - v \in L \rangle$. A trajectory τ is represented by $x^{u(\tau)}$, and two trajectory codes are algebraically equivalent when their difference lies in L .

Proposition 3.2 (Semigroup factorization of trajectory codes). *Let $\Psi : S \rightarrow R$ be a predicted algebra map into a commutative algebra of scalar hidden features, with $\Psi(x_i) = z_i$. Then Ψ factors through $\mathbb{k}[Q] \cong S/I_L$ if and only if*

$$\prod_i z_i^{u_i} = \prod_i z_i^{v_i} \quad \text{for every } u - v \in L.$$

Equivalently, every binomial in I_L has zero residual under Ψ .

Proof. By the universal property of quotient rings, Ψ factors through S/I_L exactly when $I_L \subseteq \ker \Psi$. Since I_L is generated by binomials $x^u - x^v$ with $u - v \in L$, this condition is precisely $\Psi(x^u - x^v) = 0$, which is the displayed equality. $\square$

The associated differentiable loss on a finite relation set $\mathcal{R} \subset L$ is

$$\mathcal{L}_{\text{binom}} = \frac{1}{|\mathcal{R}|} \sum_{(u,v) \in \mathcal{R}} \left(\sum_i u_i \log(z_i + \epsilon) - \sum_i v_i \log(z_i + \epsilon) \right)^2.$$

The logarithmic form avoids numerical explosion and converts products into additive constraints. It is valid when $z_i \geq 0$, which is natural for soft counts, gates, positive retrieval scores, or exponentiated affine probes.

3.2 The bits-per-byte gate

Let $D = (b_1, \dots, b_T)$ be a held-out byte stream and $q_\theta(b_t \mid b_{<t})$ the deployed predictive distribution. We write

$$\text{bits-per-byte}(q_\theta; D) = -\frac{1}{T} \sum_{t=1}^T \log_2 q_\theta(b_t \mid b_{<t}).$$

If an auxiliary certificate family C costs $B(C)$ serialized bits at deployment, its net compression utility on a held-out stream is

$$U(C) = \sum_{t=1}^T \log_2 \frac{q_{\theta, C}(b_t \mid b_{<t})}{q_\theta(b_t \mid b_{<t})} - B(C).$$

Training-only certificates have $B(C) = 0$ at deployment but can still be rejected if their distillation worsens the exported score.

Theorem 3.3 (Score-first auxiliary criterion). *A deployed auxiliary certificate or head improves the codelength of a held-out byte stream exactly when $U(C) > 0$. It improves held-out bits-per-byte exactly when*

$$\frac{1}{T} \sum_{t=1}^T \log_2 \frac{q_{\theta,C}(b_t \mid b_{<t})}{q_{\theta}(b_t \mid b_{<t})} > \frac{B(C)}{T}.$$

Proof. The codelength in bits under q_{θ} is $-\sum_t \log_2 q_{\theta}(b_t \mid b_{<t})$. With auxiliary C , the total codelength includes the serialized certificate bits and equals $B(C) - \sum_t \log_2 q_{\theta,C}(b_t \mid b_{<t})$. Subtracting the auxiliary codelength from the baseline codelength yields $U(C)$. Dividing by T gives the bits-per-byte condition. $\square$

This elementary theorem is the central engineering constraint of the paper. A Hilbert basis loss, a Scarf certificate, or a local-cohomology head can be mathematically correct and still useless for compression if it does not shift the predictive distribution enough to pay its byte cost.

3.3 Reasoning modules and filtered complexes

For each problem instance p , let $\mathcal{T}_p$ be a finite set of sampled or teacher trajectories. The multigraded trajectory module is

$$M_p = \bigoplus_{\tau \in \mathcal{T}_p} \mathbb{k} \cdot e_{\tau}, \quad \deg(e_{\tau}) = \deg(\tau),$$

with relations generated by known equivalences, verifier-identical endpoints, memory-compatible rewrites, and graph-of-thought merge identities. A filtered simplicial complex $\Delta_p(r)$ can be built on primitive factors or trajectory vertices by thresholding coactivation, verifier compatibility, retrieval compatibility, or proof-dependency scores. The chain complex $C_{\bullet}(\Delta_p(r))$ and the module M_p interact through a degree map: a simplex σ has degree $\sum_{i \in \sigma} a_i$. This is the interface between ToricGT II's persistence complexes and Part II's multigraded algebra.

Definition 3.4 (Toric algebra certificate). A toric algebra certificate for a training record is a finite object

$$\mathcal{C} = (A, Q, L, \mathcal{R}, \Delta, \beta, K, \Gamma),$$

where A is a grading group, Q a finitely generated semigroup, L a lattice of relations, $\mathcal{R} \subset L$ a finite relation basis or Markov basis fragment, Δ a finite cell or simplicial complex, β a Betti/Bass/local-cohomology target slice, K a K-polynomial or Hilbert-series fragment, and Γ metadata for quotient, saturation, or local-cohomology computations. The certificate is exact if all listed invariants are computed from the symbolic record, not predicted by the model.

4 Toric algebra and ReLU/tropical neural computation

4.1 Active regions as weight orders

A ReLU network is piecewise affine. A tropical attention head is explicitly piecewise linear: it computes maxima or minima of affine candidate scores. In either case, a hidden state h induces a weight vector $w(h)$ on candidate monomials or primitive moves. Given a lattice ideal I_L , the initial ideal $\text{in}_w(h)(I_L)$ records which monomial terms dominate the binomial relations at that hidden state. This is the algebraic shadow of an active ReLU/tropical decision.

Definition 4.1 (ReLU-toric chart). A ReLU-toric chart for a hidden table $H \in \mathbb{R}^{m \times d}$ consists of rational affine candidates $\ell_i(h) = \langle r_i, h \rangle + b_i$, exponent labels $a_i \in A$, and a relation lattice $L \subseteq \mathbb{Z}^n$. The chart assigns each token row h the weight vector $w_i(h) = \ell_i(h)$, the active set $\arg \max_i w_i(h)$, and the initial ideal $\text{in}_w(h)(I_L)$.

Proposition 4.2 (Stable active regions give stable monomial shadows). *Fix a finite relation set $\mathcal{R} \subset L$. Suppose that for a hidden state h , every binomial $x^u - x^v$ in $\mathcal{R}$ has strict weight gap*

$$|\langle w(h), u - v \rangle| \geq \gamma > 0.$$

If $\|w(h) - w(h')\|_\infty < \gamma/(2D)$, where $D = \max_{(u,v) \in \mathcal{R}} \|u - v\|_1$, then $\text{in}_w(h)(x^u - x^v) = \text{in}_w(h')(x^u - x^v)$ for all $(u, v) \in \mathcal{R}$.

Proof. For any relation (u, v) , the change in weight difference is bounded by

$$|\langle w(h) - w(h'), u - v \rangle| \leq \|w(h) - w(h')\|_\infty \|u - v\|_1 < \gamma/2.$$

Thus the sign of $\langle w, u - v \rangle$ cannot change, so the leading monomial of each binomial remains the same. $\square$

The proposition motivates a *Groebner-margin loss*

$$\mathcal{L}_{\text{init-margin}} = \frac{1}{|\mathcal{R}|} \sum_{(u,v) \in \mathcal{R}} \max\{0, \gamma - |\langle w(h), u - v \rangle|\}.$$

This is not a generic confidence penalty. It directly increases the stability of the initial monomial shadow of a toric relation set. For ReLU networks, it discourages tiny, noisy active-region changes that make reasoning brittle. For tropical attention, it makes dynamic-programming choices inspectable and stable.

4.2 Why initial degenerations help training

The book’s multigraded theory explains that K-polynomials are preserved under initial degeneration while Betti numbers can only increase. This suggests a practical curriculum. Train first on an initial monomial shadow because monomial membership, standard monomial enumeration, squarefree support, and cellular complexes are cheap. Then lift to binomial consistency once the hidden representation has stable active cells.

Theorem 4.3 (Conservative monomial-shadow curriculum). *Let M be a finitely generated multigraded module over S , and let $\text{in}_w(M)$ be an initial module under a positive weight order. Suppose a ToricGT probe estimates Betti masses from the monomial shadow $\text{in}_w(M)$. If the shadow has Betti mass above a threshold B in degrees outside an allowed region $\mathcal{D}$, then no flat lift to M can have larger homological efficiency penalty than the shadow; i.e. the shadow is a conservative upper-bound diagnostic for syzygy complexity.*

Proof. The algebraic fact is that passing to an initial module preserves the multigraded Hilbert series but cannot decrease the number of minimal syzygies in any multidegree. Thus the Betti numbers of $\text{in}_w(M)$ upper bound those of M degree-wise. Any penalty that is monotone in the offending Betti masses is therefore an upper-bound penalty. Training against the shadow may over-penalize, but it will not miss a high-complexity shadow that would signal unstable or redundant reasoning. $\square$

4.3 ReLU network applications

The ReLU/tropical applications are concrete.

1. **Active proof-path audit.** Each proof choice is a tropical maximum; the initial monomial records the selected predecessor or rule. Relation margins measure robustness.
2. **Symbolic dynamic-programming circuits.** Shortest path, alignment, parsing, and template matching are min-plus circuits; their graph-token implementation can be audited through monomial shadows.

3. **Reasoning-cell compression.** Quantized active-cell IDs plus relation margins can replace long explanation traces when the cell ID predicts future tokens.
4. **Normal-fan curriculum.** Start with coarse active-cell labels; refine to binomial relation residuals; then train Betti/local-cohomology certificates.
5. **Adversarial cell smoothing.** Penalize active-region changes not justified by changes in input graph structure, verifier state, or retrieved evidence.

5 Catalog of toric-algebraic applications, losses, metrics, techniques, and regularizers

This section contains twenty-four objectives and techniques. The first twenty-two are directly grounded in Part II of the book; the last two are connective techniques that combine multiple Part II themes. Each item is intentionally phrased as an implementable training component: source idea, mathematical formula, intended compression/reasoning mechanism, and reproducibility details.

5.1 O1. Semigroup trajectory congruence loss

Part II source. Chapter 7: semigroup rings and lattice ideals.

Mathematical object and loss. Given primitive reasoning moves with degrees a_i , build $L = \ker(\mathbb{Z}^n \rightarrow A)$ and a finite relation set $\mathcal{R} \subset L$. For positive move scores $z_i(h)$, use $\mathcal{L} = |\mathcal{R}|^{-1} \sum_{(u,v)} (\langle u, \log z \rangle - \langle v, \log z \rangle)^2$.

Why it should help ToricGT. Equivalent proof paths should induce equivalent hidden codes. This lowers conditional entropy when future tokens depend on the abstract proof state rather than on an arbitrary ordering of independent moves.

Implementation and reproducibility. Generate relation sets from small integer kernels by Smith normal form or saturation. Cache relations by task family. Evaluate on path-equivalence, proof-order, and graph-relabeling splits. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.2 O2. Saturation-hole normality regularizer

Part II source. Chapter 7: affine semigroups, Hilbert bases, saturation.

Mathematical object and loss. For a learned semigroup $Q = \mathbb{N}\{a_i\}$ and sampled lattice point b , detect $kb \in Q$ while $b \notin Q$. Penalize the score assigned to unsupported holes by $\mathcal{L}_{hole} = \mathbb{E}_b \mathbf{1}[kb \in Q, b \notin Q] s_\theta(b)$, or train a classifier for Q versus $Q^{sat} \setminus Q$.

Why it should help ToricGT. Nonnormal holes are states that appear reachable by scaling but are not actually reachable by primitive moves. In reasoning, these cause brittle interpolation: the model extrapolates a plausible intermediate proof state that no valid sequence realizes.

Implementation and reproducibility. Use Normaliz for larger semigroups; for small records solve bounded integer programs. Ablate on tasks requiring intermediate lemmas and long-context partial plans. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.3 O3. Hilbert-basis step dictionary pruning

Part II source. Chapter 7: unique minimal generators and Hilbert bases.

Mathematical object and loss. Let H_Q be the Hilbert basis of the cone generated by step degrees. Penalize primitive moves not used in any Hilbert-basis decomposition and encourage dictionary coverage: $\mathcal{L}_{HB} = \alpha|D \setminus H_Q| + \beta \sum_{h \in H_Q} \max(0, \gamma - p_\theta(h))$.

Why it should help ToricGT. A compact generator dictionary improves bits-per-byte by reducing redundant latent categories and making trajectories share codes. It also improves reasoning by forcing primitive moves to be irreducible obligations rather than arbitrary microsteps.

Implementation and reproducibility. Run after quantization stabilizes. Use soft assignments during training; export only the pruned dictionary if it improves validation. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.4 O4. Groebner-shadow curriculum

Part II source. Chapter 7 initial ideals and Chapter 8 degeneration.

Mathematical object and loss. For each lattice relation $x^u - x^v$, train first on the leading monomial under a sampled weight w , then on the full binomial. Schedule λ_{binom} from 0 to 1 after initial-shadow accuracy exceeds a threshold.

Why it should help ToricGT. Monomial shadows are easier: they are discrete active choices. Lifting to binomial relations later teaches equivalence. This matches how ReLU/tropical networks first learn active regions and then learn stable algebra inside those regions.

Implementation and reproducibility. Record weight vectors, leading terms, and shadow accuracy. Reject if the lifted relation loss improves synthetic algebra but worsens natural-text bits-per-byte. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.5 O5. Multigraded K-polynomial trajectory signature

Part II source. Chapter 8 Hilbert series and K-polynomials.

Mathematical object and loss. For a trajectory module M , compute or estimate $K(M; t) = \sum_{i,a} (-1)^i \beta_{i,a} t^a$. Train a small head $\hat{K}_\theta(\tau)$ with squared or contrastive loss against the exact finite K -slice.

Why it should help ToricGT. The K-polynomial is a small Euler-characteristic code for a complex. It can be a retrieval key that captures the shape of a proof family without storing every step.

Implementation and reproducibility. Flatten only degrees appearing in a bounded window. Use as a teacher for memory retrieval; do not add to final logits unless retrieval lift appears. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.6 O6. Betti-table reasoning regularity

Part II source. Chapter 8 multigraded Betti numbers.

Mathematical object and loss. Let $\beta_{i,a}$ be target Betti masses for the record’s resolution. Penalize off-pattern syzygies: $\mathcal{L}_\beta = \sum_{i,a} w_{i,a} |\hat{\beta}_{i,a} - \beta_{i,a}|^2 + \eta \sum_{(i,a) \notin \mathcal{D}} \hat{\beta}_{i,a}$.

Why it should help ToricGT. High off-pattern Betti mass means redundant obligations or tangled dependencies. Lowering it encourages proofs whose intermediate constraints compose cleanly.

Implementation and reproducibility. Use exact Macaulay2 for curated algebra records and synthetic small examples. For non-algebra tasks, train only coarse Betti buckets from filtered complexes. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.7 O7. Multidegree class matching

Part II source. Chapter 8 multidegrees.

Mathematical object and loss. Compute a multidegree $C(M; t)$ for exact modules or a proxy from occupied cells. Train $\hat{C}_\theta$ using cosine or Earth-mover distance over monomial terms.

Why it should help ToricGT. Multidegrees are coarser than full resolutions and therefore more robust. They can group families of equivalent reasoning trajectories across rephrasings and graph isomorphisms.

Implementation and reproducibility. Use for cross-domain retrieval: same multidegree class but different surface tokens should be positive pairs. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.8 O8. Initial-Betti upper-semicontinuity gate

Part II source. Chapter 8 Betti increase under initial ideals.

Mathematical object and loss. Compute Betti complexity on a monomial initial shadow. Gate expensive binomial or homological training only if the shadow complexity is below a budget or the task is tagged as needing high complexity.

Why it should help ToricGT. This prevents the model from wasting capacity on records whose degenerated shadow is already too tangled for the intended small artifact.

Implementation and reproducibility. Use a cheap monomial-resolution approximation. Log ‘shadow Betti budget exceeded’ separately from model failure. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.9 O9. Laurent periodic trajectory module

Part II source. Chapter 9 Laurent monomial modules.

Mathematical object and loss. Model periodic rewrites by a Laurent monomial module $M \subset \mathbb{K}[x_1^{\pm 1}, \dots, x_n^{\pm 1}]$. Train keys invariant under lattice translations $u \mapsto u + \ell$, $\ell \in L$, by contrastive loss $\|k(u) - k(u + \ell)\|^2$.

Why it should help ToricGT. Many reasoning differences are periodic or commutative. Quotienting them improves memory reuse and reduces duplicate storage.

Implementation and reproducibility. Sample lattice translations within a bounded window. Combine with graph-of-thought merge labels and retrieval positives. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.10 O10. Hull-resolution trajectory scaffold

Part II source. Chapter 9 hull resolutions.

Mathematical object and loss. Build the hull complex of a Laurent module or finite truncation; use its bounded faces as scaffold labels for trajectory dependencies. Penalize predicted boundary matrices by $\sum_k \|\hat{D}_{k-1} \hat{D}_k\|_F^2$ and optional alignment.

Why it should help ToricGT. A hull complex supplies a cellular map of syzygies. For reasoning, this is a graph of dependency cancellations: which intermediate obligations explain away which redundancies.

Implementation and reproducibility. Use only bounded finite windows. Hash the cell complex; stream sparse boundary matrices to GPU. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.11 O11. Scarf-generic minimal certificate

Part II source. Chapter 9 genericity and the Scarf complex.

Mathematical object and loss. When a lattice or module is generic, use the Scarf complex as the minimal certificate. Train a Scarf-face predictor and penalize nonunique labels that destroy Scarf minimality.

Why it should help ToricGT. Minimal certificates are ideal for compression: no redundant cells, no unnecessary syzygies. This creates compact trajectory signatures.

Implementation and reproducibility. Add small random rational perturbations to create generic synthetic curricula; keep exact nongeneric cases for robustness tests. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.12 O12. Lawrence-lift squarefree augmentation

Part II source. Chapter 9 Lawrence ideals.

Mathematical object and loss. Lift relations $u - v \in L$ to $(u - v, v - u)$ in doubled variables and use squarefree initial ideals as augmented contrastive tasks.

Why it should help ToricGT. Doubled variables separate positive and negative evidence. Squarefree shadows connect toric relations to simplicial complexes, making them useful for filtered-persistence supervision.

Implementation and reproducibility. Use for retrieval records with source/target edges or accepted/rejected proof moves. Avoid storing doubled variables at deployment unless they compress. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.13 O13. Toric quotient invariance head

Part II source. Chapter 10 toric varieties and quotients.

Mathematical object and loss. Let an abelian group G act on latent coordinates by characters. Penalize variation of invariant heads under sampled group actions: $\mathcal{L}_{quot} = \mathbb{E}_g \|f(h) - f(g \cdot h)\|^2$ while preserving equivariant side channels.

Why it should help ToricGT. Reasoning should ignore nuisance symmetries but preserve task-relevant equivariant information. Quotient losses separate the two.

Implementation and reproducibility. Apply to graph relabeling, tool-ID renaming, formatting style, and commuting independent proof steps. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.14 O14. Cox-style semistable routing window

Part II source. Chapter 10 projective quotients.

Mathematical object and loss. Assign modules to multidegrees and allow an expert or retrieval cell only if the current state is semistable for the chosen degree window. Penalize routing mass outside the semistable set.

Why it should help ToricGT. This prevents experts from firing in contexts where their algebraic assumptions are invalid. It improves reliability and behavior control.

Implementation and reproducibility. Implement as a learned boolean mask with exact symbolic audits on synthetic quotient examples. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.15 O15. Irreducible-component hallucination barrier

Part II source. Chapter 11 irreducible resolutions.

Mathematical object and loss. From an evidence semigroup ideal I , compute irreducible components $I = \cap_F I_F$. Penalize answer states that require evidence from mutually incompatible components.

Why it should help ToricGT. Hallucination often arises by combining fragments that do not share support. Irreducible components are a precise support decomposition.

Implementation and reproducibility. Use for retrieval-grounded QA: each evidence span has component labels; final claims must land in a compatible intersection. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.16 O16. Bass-number failure-mode classifier

Part II source. Chapter 11 injective resolutions and Bass numbers.

Mathematical object and loss. Train a classifier to predict Bass-like component counts $\mu_{F,a}^j$ from hidden states. Use high Bass degree as a warning for missing support or late-stage failure.

Why it should help ToricGT. Betti numbers detect generators and syzygies; Bass numbers detect where failures live. This is useful for abstention, clarification, and verifier routing.

Implementation and reproducibility. Approximate with component labels for non-algebra records; use exact injective resolutions for small monomial and semigroup examples. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.17 O17. Dualizing-complex interior evidence prior

Part II source. Chapter 12 dualizing complexes.

Mathematical object and loss. For saturated semigroups, train the model to assign high confidence only to states in the relative interior of the evidence cone; boundary states get uncertainty or request-more-evidence penalties.

Why it should help ToricGT. Interior states are supported by enough independent evidence directions. Boundary states often lack hypotheses and should not produce overconfident answers.

Implementation and reproducibility. Estimate interior membership by linear inequalities. Couple to tone/personality control: boundary states should sound cautious. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.18 O18. Ehrhart context-growth law

Part II source. Chapter 12 Ehrhart polynomials.

Mathematical object and loss. Fit the number of admissible reasoning states at budget m to an Ehrhart-like polynomial. Penalize predicted state counts that violate the learned growth law.

Why it should help ToricGT. Long-context reasoning needs budget planning. If admissible states grow polynomially in a structured family, the model should allocate retrieval and branch budget according to that law.

Implementation and reproducibility. Measure graph-of-thought branch counts at multiple budgets. Use for stopping-time policies and context-window allocation. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.19 O19. Brion vertex-cone retrieval cache

Part II source. Chapter 12 Brion’s formula.

Mathematical object and loss. Decompose a state polytope into vertex-local cone summaries. Retrieval score is a mixture over vertex caches: $r(q, m) = \max_v \{r_v(q, m) - c_v\}$ or a softmax variant.

Why it should help ToricGT. A vertex-cone cache stores local generating functions instead of every state. This is a principled compression of knowledge-graph neighborhoods and proof-state families.

Implementation and reproducibility. Index memory by vertex IDs and tangent-cone features. Evaluate recall at fixed byte budget. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.20 O20. Short rational generating-function code

Part II source. Chapter 12 short rational generating functions.

Mathematical object and loss. Encode families of reachable states by rational functions $\sum_b c_b t^b = P(t) / \prod_j (1 - t^{a_j})$ and distill coefficients or evaluations into a retrieval head.

Why it should help ToricGT. This turns many discrete states into a compact algebraic code, directly serving compression and bits-per-byte constraints.

Implementation and reproducibility. Use bounded denominators and exact integer arithmetic during certificate generation; distill to real-valued probes. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.21 O21. Cech-support local-cohomology loss

Part II source. Chapter 13 Cech complexes.

Mathematical object and loss. For an evidence ideal $I = \langle m_1, \dots, m_r \rangle$, build the Cech complex and penalize hidden states with high predicted unsupported cohomology in low degrees.

Why it should help ToricGT. Local cohomology detects information supported only on missing or forbidden loci. In reasoning, this flags answers relying on unsupported intersections of tools or memories.

Implementation and reproducibility. Use differentiable cochain maps for small r ; for large records use sampled supports. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.22 O22. Toric local cohomology coverage audit

Part II source. Chapter 13 toric local cohomology.

Mathematical object and loss. For degree b , compute the cocomplex $\nabla_Q(b)$ and its reduced homology. Train a coverage head to predict whether $H^i(\nabla_Q(b))$ is nonzero.

Why it should help ToricGT. Nonzero local cohomology marks holes in semigroup support. Coverage heads can route the model to retrieve more evidence, ask clarification, or lower confidence.

Implementation and reproducibility. Convert memory/reasoning states to degrees; compute cocomplexes for small cones; use synthetic holes as adversarial tests. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.23 O23. Cohen-Macaulay reasoning-depth objective

Part II source. Chapter 13 Cohen-Macaulay conditions.

Mathematical object and loss. Use vanishing patterns of local cohomology or regular-sequence tests as a depth criterion. Penalize reasoning modules whose low-degree cohomology persists outside expected failure slices.

Why it should help ToricGT. Cohen-Macaulayness is a strong no-hidden-hole condition. As a model objective, its proxy encourages robust coverage over all necessary reasoning dimensions.

Implementation and reproducibility. Use as audit first; train only if the proxy correlates with held-out proof success and bits-per-byte. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.24 O24. Universal grading discovery

Part II source. Chapter 8 universal gradings and lattice ideals.

Mathematical object and loss. Learn the finest stable grading that makes observed trajectory relations homogeneous. Penalize relations whose predicted degrees disagree: $\mathcal{L}_{grade} = \sum_{(u,v)} \|A_\theta u - A_\theta v\|^2$.

Why it should help ToricGT. A good grading disentangles factors such as evidence, style, tool use, proof depth, and safety. It improves controllability and retrieval.

Implementation and reproducibility. Initialize from tags; refine with low-rank integer-like matrices using straight-through rounding. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

5.25 O25. Markov-basis negative sampling

Part II source. Chapter 7 lattice ideals and integer programming.

Mathematical object and loss. Use binomial moves from a Markov basis to generate equivalent positive pairs and near-miss negative pairs. Contrastive loss separates valid lattice moves from invalid moves of similar length.

Why it should help ToricGT. This trains the model to know which rewrites preserve meaning. It improves reasoning path search and reduces spurious analogies.

Implementation and reproducibility. Generate with 4ti2 for curated tasks; for small tasks enumerate bounded kernel moves. Use the score-first protocol: report held-out bits-per-byte, task accuracy, proof/verifier metrics, retrieval recall, behavior-corridor violations, and exported artifact bytes before and after enabling this component. If the exact symbolic audit and differentiable surrogate disagree, the component remains audit-only.

6 Training paradigms derived from Part II

6.1 Paradigm A: audit-only algebraic cartography

Before training any new loss, run the current model on a diagnostic corpus and build toric algebra certificates from hidden trajectories. Quantize transition types, fit degree maps, compute small relation

lattices, estimate semigroups, generate initial ideals under active ReLU/tropical weights, and compute small Betti/K-polynomial/local-cohomology summaries. No gradient flows through these computations. The output is a map: which layers and task families already contain recoverable toric structure?

The audit reports:

1. binomial residuals on known equivalent reasoning paths;
2. semigroup holes and saturation defects;
3. Hilbert-basis coverage of primitive move dictionaries;
4. active initial-ideal stability under perturbations;
5. shadow Betti complexity;
6. Scarf uniqueness and hull-cell sparsity;
7. local-cohomology coverage holes;
8. relation between these diagnostics and held-out bits-per-byte or task errors.

6.2 Paradigm B: monomial-shadow pretraining and binomial lifting

This is the safest algebraic curriculum. Generate records with exact lattice relations, but begin by training only the initial monomial shadows. The model learns active choices: which proof predecessor, which rewrite, which evidence component, which semigroup generator. After active margins stabilize, introduce the binomial equality losses. This mirrors the algebraic fact that initial degeneration preserves Hilbert series while simplifying relations.

6.3 Paradigm C: ReLU/tropical chart regularization

For a chosen set of layers, attach low-rank rational probes whose candidate slopes define Newton polytopes and weight orders. Train for active-cell stability, relation margins, and bend consistency across graph relabelings and equivalent examples. The probes should be fake-quantized so that any exported component has predictable byte cost.

6.4 Paradigm D: resolution-shaped graph-of-thought training

For graph-of-thought trajectories, build a cell complex whose vertices are reasoning states and whose higher cells encode merge compatibility or equivalent decompositions. Train sparse boundary heads with $D_{k-1}D_k = 0$, Betti profile targets, Scarf/hull labels when available, and local-cohomology coverage labels for missing faces. Use GFlowNet rewards that favor trajectories with low algebraic residuals only when verifier accuracy and held-out bits-per-byte do not degrade.

6.5 Paradigm E: quotient and behavior control

Use toric quotient ideas to separate nuisance variables from behavior-critical variables. Nuisance transformations include graph relabeling, formatting, tool identifier names, independent step ordering, and benign paraphrase. Behavior-critical variables include safety, evidence sufficiency, uncertainty, tone, proof rigor, and compliance class. The quotient head should be invariant to the first group and equivariant or controlled with respect to the second.

6.6 Paradigm F: retrieval and memory as semigroup modules

ToricGT III treated MCPs, markdown memory, and knowledge graphs as graph-structured vector databases. The present paper adds the algebraic layer: memory states live in a semigroup module; equivalent updates differ by lattice translations; local neighborhoods are vertex-cone caches; and missing support is detected by local cohomology. Retrieval keys can include (K, β, C, ∇) -style signatures in addition to dense embeddings. A retrieval item is useful when it is close in embedding space and compatible in degree, component support, and resolution signature.

7 Implementation methodology

7.1 Data schema

Each training record should be convertible into the following JSON-like schema.

```
{
  "problem_id": str,
  "tokens": [...],
  "graph": {"nodes": [...], "edges": [...]},
  "trajectory": [step_0, ..., step_T],
  "step_types": [i_0, ..., i_{T-1}],
  "degrees": [[...], ...],
  "relations": [[u, v], ...],
  "filtered_complex": {"simplices": [...], "filtration": [...]},
  "certificates": {
    "semigroup": A_matrix,
    "lattice_kernel": L_basis,
    "hilbert_basis": H,
    "initial_terms": {...},
    "betty_slice": {...},
    "local_cohomology_slice": {...}
  },
  "targets": {"answer": ..., "verifier": ..., "behavior": ...}
}
```

The schema separates exact certificate generation from differentiable training. Exact computation can be slow and CPU-bound; it is cached by a content hash of $(A, L, \Delta, \text{task tags})$. GPU batches receive compact tensors: relation indices, sparse boundary matrices, degree labels, component labels, and target vectors.

7.2 Certificate generators

For small examples, pure Python is sufficient.

1. **Kernel and lattice basis:** compute Smith normal form or use integer row reduction. For bounded kernels, enumerate moves $u - v$ with $\|u\|_1 + \|v\|_1 \leq D$.
2. **Semigroup membership:** solve a bounded nonnegative integer feasibility problem. For tiny cases, dynamic programming over degrees works.
3. **Hilbert basis:** enumerate indecomposable elements in a bounded cone slice, or call Normaliz for real workloads.

4. **Initial ideals:** given relation set and weight vector, select leading monomials.
5. **Free resolutions:** for curated algebra examples, call Macaulay2. For generic Scarf examples, construct the Scarf complex directly from unique least common multiples or Laurent labels.
6. **Local cohomology:** construct Cech or Ishida-style complexes for small support ideals; compute homology by sparse linear algebra over a finite field.

7.3 GPU integration

A minimal PyTorch interface has three parts.

```
class ToricAlgebraCertificate:
    relation_u: LongTensor # [r, n]
    relation_v: LongTensor # [r, n]
    degrees: LongTensor    # [n, d]
    boundary: list[SparseTensor]
    betti_target: FloatTensor
    component_labels: LongTensor
    local_cohomology_target: FloatTensor

class ToricAlgebraProbe(nn.Module):
    def forward(self, hidden, cert):
        z = positive_step_scores(hidden)
        binom = binomial_residual(z, cert.relation_u, cert.relation_v)
        beta_hat = betti_head(hidden)
        comp_hat = component_head(hidden)
        local_hat = local_cohomology_head(hidden)
        return binom, beta_hat, comp_hat, local_hat
```

The expensive symbolic objects never need to be materialized as large dense tensors. Relation matrices are sparse. Boundary matrices are sparse. Betti/local-cohomology targets are small slices. This matches the ToricGT philosophy: training-only structure, low-rank probes, exact audits, and score-first export.

7.4 Quantization and artifact accounting

A toric-algebraic head is exportable only when its compressed codelength is justified. Low-rank factorization is the default:

$$W = UV^\top, \quad U \in \mathbb{R}^{m \times r}, \quad V \in \mathbb{R}^{d \times r}, \quad r \ll d.$$

Fake quantization during training estimates the exported behavior. Every component must report:

$$B_{\text{comp}} = B_{\text{weights}} + B_{\text{lookup}} + B_{\text{code}} + B_{\text{metadata}}.$$

A component survives export only if its held-out log-score lift exceeds B_{comp} in bits, or if the project is in a research-model mode where a predeclared reasoning metric is allowed to improve without bits-per-byte gain.

7.5 Reproducible evaluation

Use clustered splits by algebraic family, not random row splits. A relation family in training should not reappear with only renamed variables in validation. Report:

1. held-out bits-per-byte on ordinary prose and structured graph/proof serializations;
2. exact answer accuracy and verifier agreement;
3. equivariance error under graph relabeling;
4. binomial residual on unseen lattice moves;
5. saturation-hole classification accuracy;
6. Hilbert-basis coverage and generator sparsity;
7. initial-shadow stability under perturbation;
8. Betti/K-polynomial/multidegree prediction error;
9. Scarf/hull certificate precision and recall;
10. local-cohomology coverage error;
11. behavior-corridor violations and calibration.

No metric should be interpreted in isolation. A model with excellent algebraic diagnostics but worse held-out bits-per-byte is not a better compressed model. It may still be a useful teacher if it improves proof validity or out-of-distribution algebraic reasoning and distills into a smaller student.

8 Worked examples

8.1 A commuting-proof semigroup

Suppose a toy proof has three primitive moves: x_1 retrieves a lemma, x_2 applies it, and x_3 formats the conclusion. If retrieval and formatting commute but applying the lemma must occur after retrieval, the multiset relation might include only

$$x_1x_3 - x_3x_1 = 0$$

in a noncommutative trace model, but after abelianization the degree relation says $e_1 + e_3 = e_3 + e_1$, which is trivial. This example shows why ToricGT needs typed semigroups: the abelian semigroup captures reusable count structure, while the graph-of-thought DAG captures order constraints. A useful certificate therefore stores both Q and a partial order on moves.

8.2 A binomial relation for equivalent decomposition

Let primitive moves have degrees

$$a_1 = (1, 0), \quad a_2 = (0, 1), \quad a_3 = (1, 1).$$

Then $a_1 + a_2 = a_3$, and $I_L = \langle x_1x_2 - x_3 \rangle$. In reasoning terms, doing two elementary steps should match a single macro-step. A model with positive scores (z_1, z_2, z_3) should satisfy

$$\log z_1 + \log z_2 - \log z_3 \approx 0.$$

If not, the hidden state treats macro and micro reasoning inconsistently. On byte prediction, this inconsistency can raise entropy: after a prefix that admits either decomposition, the model does not confidently predict the same continuation.

8.3 A saturation hole

Let $Q = \mathbb{N}\{(2, 0), (0, 2), (1, 1)\} \subset \mathbb{Z}^2$. The point $(1, 0)$ is not in Q , and no multiple $k(1, 0)$ with odd k lies in Q , but $2(1, 0) = (2, 0) \in Q$. Thus $(1, 0)$ is a saturation hole. As a reasoning analogy, a model may learn that two copies of a subclaim can be proven but fail to represent a single copy. Penalizing confidence on such holes encourages valid decomposition at the correct scale.

8.4 A Scarf certificate for generic relation data

Consider a small generic lattice ideal whose Scarf complex is minimal. The training record stores only the faces of the Scarf complex and the signed boundary maps. The model predicts a boundary $\hat{D}$ from hidden states, and the local loss $\|\hat{D}_{k-1}\hat{D}_k\|_F^2$ checks chain-complex consistency. If the Scarf complex is minimal, every cell is necessary; deleting a cell means the model has lost a required dependency. This makes Scarf certificates attractive for compact reasoning audits.

8.5 Local cohomology as evidence-hole detection

Suppose evidence sources define a semigroup Q of supported combinations. A degree b represents the evidence profile needed for an answer. If the cocomplex $\nabla_Q(b)$ has nonzero cohomology, then b lies in a hole or unsupported support pattern. The model should either retrieve more evidence, ask a clarifying question, or lower confidence. This is a sharper signal than a generic uncertainty score because it names the combinatorial source of missing support.

9 Additional theoretical guarantees

Theorem 9.1 (Lattice-translation invariant retrieval). *Let memory keys be functions $k : \mathbb{Z}^n \rightarrow \mathbb{R}^d$ satisfying $k(u + \ell) = k(u)$ for all $\ell \in L$. Then retrieval scores of the form $r(u, v) = \langle k(u), k(v) \rangle$ descend to the quotient $\mathbb{Z}^n/L$. Conversely, any score on $\mathbb{Z}^n/L$ can be lifted to an L -translation invariant score on $\mathbb{Z}^n$.*

Proof. If $u' = u + \ell$ and $v' = v + \ell'$, then $k(u') = k(u)$ and $k(v') = k(v)$, so $r(u', v') = r(u, v)$. Thus the score is well-defined on equivalence classes. Conversely, given $\bar{r}$ on the quotient, define $r(u, v) = \bar{r}([u], [v])$. This is invariant by construction. $\square$

Theorem 9.2 (Relation-margin robustness). *Let $z_i(h) > 0$ be positive differentiable scores with $|\log z_i(h) - \log z_i(h')| \leq \epsilon$ for all i . For a relation (u, v) , define*

$$R_{u,v}(h) = \langle u - v, \log z(h) \rangle.$$

Then $|R_{u,v}(h) - R_{u,v}(h')| \leq \epsilon \|u - v\|_1$. In particular, if $|R_{u,v}(h)| > \epsilon \|u - v\|_1$, the sign of the relation residual cannot change under perturbation from h to h' .

Proof. The bound follows from Holder's inequality:

$$|\langle u - v, \log z(h) - \log z(h') \rangle| \leq \|u - v\|_1 \|\log z(h) - \log z(h')\|_\infty \leq \epsilon \|u - v\|_1.$$

The sign statement is immediate. $\square$

Theorem 9.3 (Sparse certificate advantage). *Let a certificate C define a partition of validation contexts into cells $\mathcal{S}$. Suppose the deployed model without C uses distribution q_0 , and the model with C uses the cell-conditional mixture $q_C(b \mid c) = q_s(b \mid c)$ when $c \in s$. If*

$$\sum_{s \in \mathcal{S}} \sum_{t: c_t \in s} \log_2 \frac{q_s(b_t \mid c_t)}{q_0(b_t \mid c_t)} > B(C),$$

then the certificate improves held-out compression. If the partition is independent of future validation bytes and computed only from prefixes or fixed side information, the comparison is score-valid.

Proof. The first statement is the score-first auxiliary criterion applied after grouping terms by cell. The validity statement follows because the cell choice does not depend on the byte being scored or any future byte; therefore it defines a normalized prefix-causal predictive distribution. $\square$

Theorem 9.4 (Local-cohomology warning rule). *Let a finite evidence certificate assign a degree b to a proposed answer and let $H^i(\nabla_Q(b); \mathbb{k}) \neq 0$ for some forbidden degree range $i < d_0$. Suppose the policy maps nonzero forbidden cohomology to either retrieval, clarification, or low-confidence decoding. Then the policy cannot output a high-confidence answer on that certificate unless the local-cohomology surrogate is wrong or the warning gate is disabled.*

Proof. This is a soundness statement about the finite policy, not a theorem about truth in the world. The gate condition is a deterministic implication from the computed certificate to the allowed action set. If the exact certificate has forbidden cohomology, high-confidence answer actions are masked. Therefore such an answer can occur only when the model uses an incorrect surrogate in place of the exact certificate or the mask is not applied. $\square$

Remark 9.5. The last theorem illustrates the appropriate style of guarantee for learning systems. The algebra can certify a property of the internal finite record. It cannot certify that the external world matches the record. Grounding still requires source verification and data hygiene.

10 A concrete experimental plan

10.1 Synthetic algebra suite

Build a suite with increasing difficulty:

1. small semigroups in $\mathbb{Z}^2$ and $\mathbb{Z}^3$ with known Hilbert bases;
2. saturated versus unsaturated semigroups with labeled holes;
3. lattice ideals from random small matrices and curated nongeneric examples;
4. generic Scarf examples and nongeneric counterexamples;
5. Lawrence lifts with squarefree initial ideals;
6. Ehrhart counting tasks for small polytopes;
7. local-cohomology cocomplex classification tasks.

Every example stores exact certificates. Splits are by semigroup family and matrix distribution, not by random row.

10.2 Reasoning suite

Convert the algebraic objects into reasoning tasks:

1. proof-path equivalence under commuting independent lemmas;
2. macro-step versus micro-step decomposition;
3. retrieval of a helper proof with same Betti/K signature;

4. detection of missing evidence holes;
5. controlled style: boundary evidence should produce cautious tone;
6. ReLU/tropical dynamic-programming tasks with active initial-ideal labels;
7. graph-of-thought merge consistency under branch recombination.

10.3 Ablations

The minimal ablation table is:

Ablation	Question
No toric algebra certificates	Does the base ToricGT representation already learn the relation?
Only monomial shadows	Are active choices enough without binomial equivalence?
Binomial loss without saturation	Are holes responsible for failures?
Hilbert-basis pruning disabled	Does dictionary redundancy hurt bits-per-byte?
No Betti/K signature in retrieval	Does homological retrieval beat embedding-only retrieval?
No local cohomology	Does coverage detection reduce hallucination or overconfidence?
No quotient invariance	Are nuisance symmetries leaking into reasoning?
Training-only certificates, no export	Is the benefit distillable?
Exported low-rank head	Does the compressed head pay for its own byte cost?

10.4 Stopping rules

Stop or disable an auxiliary if any of the following occurs:

1. held-out bits-per-byte worsens beyond the predeclared tolerance on ordinary prose;
2. exact symbolic audits disagree with the differentiable surrogate on a random certificate sample;
3. the auxiliary improves synthetic algebra but harms natural reasoning or behavior calibration;
4. the exported model loses the improvement seen in floating point;
5. the component introduces non-prefix-causal validation dependence.

10.5 Expected failure modes

Overfitting algebraic toys. The model may learn to solve exact semigroup examples while harming general language modeling. Use tagged slice metrics and never promote solely on synthetic gains.

Bad quantization of degrees. If the hidden step quantizer is unstable, all downstream algebra becomes noise. Run audit-only quantizer stability tests before algebraic training.

Incorrect finite relation basis. A small relation set may miss important equivalences. Track residuals on held-out relations and use Markov-basis sampling when available.

Surrogate mismatch. A neural Betti/local-cohomology head can become a decorative classifier. Periodically recompute exact certificates and compare.

Behavior overconstraint. Algebraic support constraints should not force robotic tone or refusal. Couple component warnings to calibrated behavior policies, not rigid final text templates.

11 Expanded theory: why toric algebra can change learning, not only diagnostics

This section records the mathematical mechanism behind the proposed objectives. The common pattern is: choose a finite certificate object from Part II of the book, attach it to a bounded reasoning record, prove that the certificate is invariant under an intended equivalence of trajectories, and train a small differentiable probe to predict or respect that certificate. The probe is useful for a language or graph model only when it changes the conditional distribution assigned to future bytes, tokens, nodes, or proof obligations. Thus the mathematical invariants below are never promoted merely because they are elegant; they are promoted when they reduce conditional entropy or improve a predeclared reasoning metric under a score-first protocol.

11.1 A finite toric algebra certificate functor

Let $\mathcal{R}$ be a bounded family of reasoning records. A record is a tuple

$$r = (G, H, \tau, E, V, \mathcal{F}, y),$$

where G is a graph-of-thought, $H = (h_v)_{v \in V(G)}$ are hidden states, τ is a typed edge relation, E is retrieved evidence, V is verifier metadata, $\mathcal{F}$ is an optional filtered simplicial complex at every reasoning vertex, and y is the supervised target. A *Part-II certificate functor* is a map

$$\mathbf{C} : \mathcal{R} \longrightarrow \mathbf{Cert}_{\text{toric}}$$

whose output consists of a finite subset of the following data:

$$(Q, L, H_Q, \mathcal{B}, \text{in}_w I_L, K, \beta, \Delta_{\text{hull}}, \Delta_{\text{Scarf}}, \mathcal{I}, \Omega_Q^\bullet, \nabla_Q(\cdot)).$$

Here Q is a semigroup of discrete reasoning increments, L is the lattice of trajectory congruences, H_Q is the minimal semigroup dictionary or Hilbert basis in the saturated case, $\mathcal{B}$ is a finite binomial relation set, $\text{in}_w I_L$ is an initial monomial shadow, K is a K-polynomial or a low-degree truncation, β is a Betti profile, Δ_{hull} and Δ_{Scarf} are cellular supports for syzygies, $\mathcal{I}$ is an irreducible or primary support decomposition, $\Omega_Q^\bullet$ is a dualizing or injective complex when available, and $\nabla_Q(b)$ is the local-cohomology cocomplex in degree b . Every object in this list is finite after bounded truncation.

Definition 11.1 (Certificate-faithful probe). A probe P_θ attached to a ToricGT hidden state is ϵ -certificate-faithful on a validation family $\mathcal{V}$ if

$$\frac{1}{|\mathcal{V}|} \sum_{r \in \mathcal{V}} d_{\mathbf{C}}(P_\theta(H(r)), \mathbf{C}(r)) \leq \epsilon,$$

where $d_{\mathbf{C}}$ is an exact certificate metric: Hamming distance for support and initial-ideal labels, Frobenius residual for boundary matrices, edit distance for cellular face posets, Wasserstein distance for multigraded Betti mass, or relative cohomology mismatch for local-cohomology cocomplexes.

Definition 11.2 (Predictive transfer). A certificate family has positive predictive transfer at artifact cost B when the exported model $q_{\theta, B}$ satisfies

$$\Delta_{\text{byte}}(\mathbf{C}) = \frac{1}{N} \sum_{t=1}^N \log_2 \frac{q_0(b_t \mid b_{<t})}{q_{\theta, B}(b_t \mid b_{<t})} > 0,$$

under strict prefix dependence and after serialized artifact accounting. For graph reasoning, replace the next byte by the next audited graph symbol, proof step, or verifier-labeled action and report the corresponding cross-entropy lift separately.

Theorem 11.3 (Certificate invariance under trajectory congruence). *Let $\pi : T \rightarrow Q$ map a free monoid of primitive reasoning moves to a commutative semigroup Q . Suppose two trajectories $\gamma, \gamma' \in T$ have exponent vectors $u, v \in \mathbb{N}^n$ with $u - v \in L$, where $L = \ker(\mathbb{Z}^n \rightarrow \mathbf{gp}(Q))$. Then every certificate that factors through $k[Q] \cong \mathbb{k}[x_1, \dots, x_n]/I_L$ assigns the same semigroup degree to γ and γ' . In particular, K -polynomial signatures, multigraded Hilbert degrees, semigroup-degree Betti labels, quotient-invariant retrieval keys, and toric quotient heads are invariant under this congruence.*

Proof. The congruence $u - v \in L$ is exactly the relation $x^u - x^v \in I_L$. The quotient presentation $k[Q] \cong S/I_L$ identifies x^u and x^v with the same basis element $t^{\pi(u)} = t^{\pi(v)}$. Any invariant constructed from the Q -grading, or from a free resolution of S/I_L whose basis elements are labeled by Q -degrees, therefore depends only on the common class of u and v . The listed certificates are all functions of the quotient degree or of modules over the quotient ring, so they are unchanged. $\square$

11.2 Minimum-description interpretation of Hilbert bases

A central compression use of Hilbert bases is dictionary selection. Let $Q = C \cap A$ be pointed and saturated, and let H_Q be its Hilbert basis. Suppose a reasoning state is represented by a nonnegative code vector $z \in \mathbb{N}^{|D|}$ over a dictionary $D \subset Q$, with decoded semigroup element $\sum_i z_i d_i$. A dictionary is *complete up to radius R* if every $q \in Q$ with height $\omega(q) \leq R$ is representable by nonnegative combinations of dictionary elements.

Proposition 11.4 (Hilbert-basis necessity). *If D is complete up to every radius R , then $H_Q \subseteq D$. Consequently a deployable dictionary for exact semigroup steps cannot omit a Hilbert-basis element without losing some reachable state.*

Proof. Let $h \in H_Q$. If $h \notin D$ and D generates all of Q , then $h = \sum_i z_i d_i$ for nonzero z . Since all $d_i \in Q \setminus \{0\}$, this expresses h as a nontrivial semigroup sum, contradicting irreducibility of h . Therefore every complete dictionary must contain h . Since h has finite height, omitting it already breaks completeness at radius $R = \omega(h)$. $\square$

Corollary 11.5 (Pruning rule). *If a learned reasoning-step dictionary contains a code $d \notin H_Q$ and $d = h_1 + \dots + h_m$ with $h_i \in H_Q$, then storing d is useful only if it shortens runtime or improves prediction enough to pay its artifact byte cost. It is not needed for exact reachability.*

This corollary gives a concrete bits-per-byte experiment. Train one model with the full learned dictionary and another in which composite semigroup moves are replaced by sequences of Hilbert-basis moves plus recurrence. If the latter preserves validation loss while reducing artifact bytes, the Hilbert-basis representation is preferred. If the macro-step improves long reasoning enough to compensate for its stored cost, the macro-step is retained as a cached derived generator rather than as a primitive.

11.3 Initial ideals as ReLU and tropical shadows

For a lattice ideal $I_L \subset S$ and a weight w , the initial ideal $\text{in}_w(I_L)$ records which monomial side of each binomial relation wins under w . A ReLU or tropical subnetwork similarly selects active affine pieces. The two notions meet by using a probe that maps a hidden state h to a weight vector $w(h) \in \mathbb{R}^n$.

Definition 11.6 (Initial-shadow loss). Let $\mathcal{B} \subset I_L$ be a finite relation set, with $b_j = x^{u_j} - x^{v_j}$. Given a hidden state h , set

$$s_j(h) = \langle w(h), u_j - v_j \rangle.$$

If the audited teacher initial side is $\ell_j \in \{+1, -1, 0\}$, where 0 means tie or unresolved, define

$$\mathcal{L}_{\text{init}}(h) = \sum_{j:\ell_j \neq 0} \log(1 + \exp(-\ell_j s_j(h))) + \lambda_{\text{tie}} \sum_{j:\ell_j=0} s_j(h)^2.$$

Theorem 11.7 (Piecewise-linear stability of initial shadows). *Assume $\min_j |s_j(h)| \geq \gamma > 0$ and $\|w(h) - w(h')\|_\infty \leq \delta$. If $\delta \max_j \|u_j - v_j\|_1 < \gamma$, then h and h' have the same initial side for every relation in $\mathcal{B}$.*

Proof. For each relation,

$$|s_j(h) - s_j(h')| \leq \|w(h) - w(h')\|_\infty \|u_j - v_j\|_1 \leq \delta \max_j \|u_j - v_j\|_1 < \gamma.$$

Thus no score can cross zero. The sign of s_j , and therefore the chosen initial monomial, is unchanged. $\square$

The stability theorem is a direct training rule: margin is not an aesthetic quantity. It bounds whether a small hidden perturbation changes the algebraic normal form. For reasoning agents this is useful because a hidden trajectory with low initial-shadow margin is brittle: tiny context changes can flip which proof relation is used, which retrieved fact is dominant, or which tool call is treated as canonical.

11.4 K-polynomials as compact multi-step signatures

The K-polynomial appears as an alternating sum over a free resolution. In training, this supplies a low-byte signature for multi-step reasoning. Let $F_\bullet$ be a finite certificate resolution with basis labels $a_{i,j} \in A$. Its truncated K-signature is

$$K_{\leq D}(F_\bullet) = \sum_{i,j:\|a_{i,j}\| \leq D} (-1)^i t^{a_{i,j}}.$$

Proposition 11.8 (Additivity under decomposed reasoning). *Suppose a reasoning problem decomposes into two independent certificate modules M_1, M_2 with a short exact sequence*

$$0 \rightarrow M_1 \rightarrow M \rightarrow M_2 \rightarrow 0.$$

Then the exact K-polynomial satisfies $K(M) = K(M_1) + K(M_2)$. Therefore a learned K-signature head can be trained by additivity over graph-of-thought branch composition.

Proof. Hilbert series are additive on short exact sequences. Multiplying by the common denominator defining the K-polynomial preserves additivity. Equivalently, choose free resolutions and take the Euler characteristic of multigraded free modules; Euler characteristics are additive in exact triangles. $\square$

For ToricGT this gives a branch-merge consistency loss. If branches b_1, b_2 are independent and merge into m , require

$$\mathcal{L}_{K\text{-merge}} = \left\| \widehat{K}(m) - \widehat{K}(b_1) - \widehat{K}(b_2) \right\|_1.$$

The loss is computationally cheap because the signature is a sparse vector over degrees. It is particularly appropriate for long-context settings: a 1–10M token context can contain many proof branches, but the K-signature of each branch can be stored as a small sparse multiset of degrees.

11.5 Local cohomology as coverage and uncertainty

Local cohomology measures support failure. In the training setting, this means it can diagnose when a reasoning trajectory has a hole: a missing evidence region, an uncovered tool permission, an omitted boundary case, or a sparse retrieval neighborhood that should have been filled before giving a confident answer.

Definition 11.9 (Coverage cocomplex of a reasoning state). Let Q be the semigroup of certified evidence increments and let $b \in \mathbb{Z}^d$ be the target degree of a reasoning state. Define

$$\nabla_Q^{\text{reason}}(b) = \{F \subseteq Q \text{ a face-like support class} : b \in Q - F\}.$$

If an exact semigroup is not available, approximate $Q - F$ using retrieved evidence cones and active face labels. The local-cohomology coverage score is

$$\text{Hole}_i(b) = \dim \tilde{H}_i((\nabla_Q^{\text{reason}}(b))^\vee; \mathbb{k}).$$

Theorem 11.10 (Hole penalty is monotone under evidence completion). *Assume adding evidence enlarges the semigroup support from Q to $Q' \supseteq Q$ without changing the ambient face poset. Then $\nabla_Q^{\text{reason}}(b) \subseteq \nabla_{Q'}^{\text{reason}}(b)$. Thus every newly added evidence item can only add cells to the coverage cocomplex. In particular, a persistent nonzero reduced homology class after evidence completion identifies a structural rather than accidental coverage hole.*

Proof. If F satisfies $b \in Q - F$, then $b = q - f$ for some $q \in Q$ and $f \in F$. Since $Q \subseteq Q'$, the same representation shows $b \in Q' - F$. Hence the cocomplex only grows. Nonzero homology after repeated additions cannot be blamed on an item that was available but omitted; it reflects the topology of the remaining uncovered region or a mismatch between the chosen faces and the target degree. $\square$

This theorem yields a pragmatic behavior policy. If a high-confidence answer is produced while $\text{Hole}_i(b) > 0$ for some critical i , lower the confidence corridor, ask for clarification, retrieve more evidence, or emit a qualified answer. The target is not to make the model timid. The target is to distinguish ordinary uncertainty from topologically structured missing support.

12 Detailed Part-II-derived objective cards

The following cards expand the catalog into reproducible research units. Each card gives the audit object, differentiable surrogate, expected model benefit, and a minimal ablation. These are intended to be implemented independently; combining them all at once would make attribution impossible.

Card A: lattice-ideal normal-form head. Audit object: a finite Groebner or Markov relation table $\mathcal{B} \subset I_L$. Surrogate: predict a canonical representative $\text{NF}(u)$ of a move exponent vector u . Loss:

$$\mathcal{L}_{\text{NF}} = \text{CE}(\widehat{\text{NF}}(u), \text{NF}(u)) + \lambda \sum_{x^a - x^b \in \mathcal{B}} \|z(a) - z(b)\|_2^2.$$

Benefit: reduces duplicate hidden states for equivalent proof paths and improves retrieval precision. Ablation: remove the relation term while keeping the same canonical-representative labels.

Card B: saturation-hole detector. Audit object: holes $Q^{\text{sat}} \setminus Q$ in bounded degree. Surrogate: binary classifier $p_{\text{hole}}(b \mid h)$. Loss:

$$\mathcal{L}_{\text{hole}} = -y_b \log p_{\text{hole}} - (1 - y_b) \log(1 - p_{\text{hole}}) + \lambda y_b \max(0, c(h) - c_{\text{max}})^2,$$

where $c(h)$ is confidence. Benefit: prevents confident extrapolation through missing evidence. Ablation: keep the classifier but decouple it from confidence and behavior corridors.

Card C: Hilbert-basis macro/micro controller. Audit object: decompositions of reachable moves into Hilbert-basis elements. Surrogate: a controller decides whether to emit a macro move or a sequence of primitives. Loss:

$$\mathcal{L}_{\text{Hctrl}} = \mathcal{L}_{\text{task}} + \lambda_{\text{len}} \mathbb{E}[T_{\text{steps}}] + \lambda_B B_{\text{macro}} - \lambda_{\text{lift}} \Delta_{\text{byte}}.$$

Benefit: discovers when storing a reusable macro is worth bytes and when recurrence suffices. Ablation: force all macro moves off.

Card D: initial-shadow active-region regularizer. Audit object: $\text{in}_w(I_L)$ labels under teacher weights. Surrogate: hidden-to-weight map $w_\theta(h)$. Loss is $\mathcal{L}_{\text{init}}$ above plus a margin term. Benefit: aligns ReLU/tropical active regions with algebraic normal forms. Ablation: randomize relation labels; performance should collapse if the objective is genuinely algebraic.

Card E: Betti sparsity and regularity. Audit object: multigraded Betti table $\beta_{i,a}$. Surrogate: sparse Betti head. Loss:

$$\mathcal{L}_\beta = \text{BCE}(\widehat{\text{supp}} \beta, \text{supp } \beta) + \lambda \sum_{i,a} \widehat{\beta}_{i,a} (|a| - i - r_0)_+.$$

Benefit: biases trajectories toward resolutions with predictable dependency depth. Ablation: replace Betti labels by total trajectory length labels and compare OOD proof generalization.

Card F: hull-cell path supervision. Audit object: cell path in a hull resolution of a Laurent monomial module. Surrogate: sequence of predicted cell ids. Loss combines cell CE with adjacency penalties:

$$\mathcal{L}_{\text{hullpath}} = \sum_t \text{CE}(\hat{c}_t, c_t) + \lambda \sum_t \mathbf{1}[(\hat{c}_t, \hat{c}_{t+1}) \notin E_{\text{hull}}].$$

Benefit: teaches the model to traverse syzygy geometry rather than treating every proof step as an unrelated token. Ablation: same number of labels but shuffled cell adjacency.

Card G: Scarf uniqueness probe. Audit object: whether an exponent label is unique among least common multiples or lattice-module labels. Surrogate: uniqueness probability p_{unique} . Loss:

$$\mathcal{L}_{\text{Scarf}} = \text{BCE}(p_{\text{unique}}, y_{\text{unique}}) + \lambda H(p_{\text{cell}} \mid y_{\text{unique}} = 1).$$

Benefit: identifies steps whose explanations are canonical and therefore good candidates for memory. Ablation: retrieve memories by final embedding only.

Card H: Lawrence squarefree lift. Audit object: the Lawrence lift $\Lambda(L)$ and its squarefree initial shadows. Surrogate: split each feature into positive/negative channels and penalize non-squarefree coactivation. Loss:

$$\mathcal{L}_{\text{Law}} = \sum_i (p_i^+ p_i^-)^2 + \lambda \mathcal{L}_{\text{init}}^{\Lambda(L)}.$$

Benefit: gives a safe binary augmentation for ambiguous directions and sign-sensitive reasoning. Ablation: remove the positive/negative split.

Card I: irreducible support decomposition. Audit object: irreducible components of monomial ideals in $k[Q]$. Surrogate: support classifier over face-indexed components. Loss:

$$\mathcal{L}_{\text{irr}} = \sum_F \text{BCE}(\hat{y}_F, y_F) + \lambda \text{TV}(\hat{y}, \text{face order}).$$

Benefit: separates distinct failure modes that ordinary scalar confidence merges. Ablation: train a single scalar error classifier.

Card J: Bass-warning head. Audit object: Bass numbers $\mu_{F,a}^j(M)$ along faces and degrees. Surrogate: low-rank face-degree tensor predictor. Loss:

$$\mathcal{L}_{\text{Bass}} = \sum_{j,F,a} w_{j,F,a} \left| \hat{\mu}_{F,a}^j - \mu_{F,a}^j \right|.$$

Benefit: detects embedded or delayed failure support, especially when a generated proof appears locally valid but has a hidden boundary defect. Ablation: replace Bass labels by Betti labels to test whether injective-side information is distinct.

Card K: Ehrhart growth regulator. Audit object: counted reachable states in dilated polytopes or filtered context windows. Surrogate: predicted count polynomial or quasi-polynomial. Loss:

$$\mathcal{L}_{\text{Ehr}} = \sum_{m \leq M} \left| \log(1 + \hat{E}(m)) - \log(1 + E(m)) \right|^2.$$

Benefit: learns how many states a reasoning process should create as context grows, preventing both underexploration and combinatorial explosion. Ablation: replace Ehrhart targets by raw token counts.

Card L: Brion vertex-cache allocation. Audit object: vertex-cone decomposition of a generating function. Surrogate: choose which local cache entries to allocate around vertices of the active polytope. Loss:

$$\mathcal{L}_{\text{Brion}} = \mathcal{L}_{\text{pred}} + \lambda \sum_v B_v - \eta \sum_v \text{lift}_v,$$

with byte cost B_v and measured lift from vertex-local retrieval. Benefit: compresses long-context retrieval by storing local rational pieces instead of a large flat table. Ablation: uniform cache allocation.

Card M: Cech and Ishida coverage. Audit object: Cech/Ishida complex for support on an ideal or semigroup maximal support. Surrogate: cochain residual and cohomology-rank head. Loss:

$$\mathcal{L}_{\text{cov}} = \sum_i \|\hat{d}_{i+1}\hat{d}_i\|_F^2 + ho \sum_i |\widehat{\dim H^i} - \dim H^i|.$$

Benefit: turns evidence coverage into a chain-complex problem. Ablation: a heuristic retrieval-count threshold.

13 Reproducibility appendix: data, code paths, and exact audits

13.1 Canonical record format

Every synthetic or curated record should be serialized as line-delimited JSON with the following minimal schema:

```
{
  "id": "semigroup-family/split/hash",
  "prompt": "natural-language or graph task prompt",
  "graph": {"nodes": [...], "edges": [...], "types": [...]},
  "trajectory": [{"move": "x3*x5", "degree": [1,0,2], "hidden_slot": 4}],
  "certificate": {
    "A": [...],
    "L_basis": [...],
  }
}
```



```

    "relations": [{"u": [...], "v": [...]}],
    "hilbert_basis": [...],
    "initial_labels": [...],
    "beti": [{"i": 0, "deg": [...], "value": 1}],
    "cells": {"vertices": [...], "faces": [...]},
    "local_cohomology": [{"degree": [...], "cohomology": [...]}]
  },
  "target": {"answer": "...", "verifier": true, "bytes": 123}
}

```

The record hash is computed from the exact certificate, not from the surface prompt. This prevents leakage in which train and validation share the same semigroup under different prose.

13.2 Exact certificate generation

Certificate generation is CPU-side. The default pipeline is:

1. Sample or load a small integer matrix A with columns a_i .
2. Compute $L = \ker_{\mathbb{Z}}(A)$ by Smith normal form.
3. Generate a bounded binomial relation set and saturate by the product of variables when constructing I_L .
4. Compute Hilbert-basis or saturation-hole data for bounded cones using Normaliz, 4ti2, Sage, Macaulay2, or a verified custom routine for small dimensions.
5. Compute initial-shadow labels under sampled weights w .
6. For small examples, compute multigraded Betti tables and cellular supports by Macaulay2; for large examples, store only bounded-degree targets.
7. Construct local-cohomology cocomplexes by enumerating faces and checking membership in $Q - F$.
8. Serialize sparse tensors in coordinate format and cache by SHA256.

No symbolic computation should run inside the GPU training step. The GPU receives dense hidden states, sparse boundary matrices, integer labels, and compressed target vectors.

13.3 Deterministic validation and leakage control

The validation protocol must be score-first. It must also be algebra-split. A split by random examples is insufficient because many different prompts can encode the same lattice ideal or the same semigroup holes. Use the following split keys:

$$\text{split_key} = \text{hash}(\text{SNF}(A), \text{Hilb}(Q)_{\leq R}, \text{relation support}, \text{family tag}).$$

All records with the same key go to the same split. For curated reasoning data, add a text-source key and an answer-template key. Report results on synthetic algebra, translated natural language, graph JSON, proof text, and ordinary prose separately.

13.4 Recommended hyperparameters

A conservative first run should use tiny weights:

$$\lambda_{\text{init}} = 10^{-4}, \quad \lambda_{\beta} = 10^{-4}, \quad \lambda_{\text{hole}} = 10^{-4}, \quad \lambda_{\text{cov}} = 10^{-5}, \quad \lambda_{\text{quotient}} = 10^{-4}.$$

After probe-only training reaches high certificate accuracy, increase only one family at a time. The activation order should be

initial shadows $\rightarrow$ semigroup congruence $\rightarrow$ Hilbert basis $\rightarrow$ Betti/K signatures
 $\rightarrow$ Scarf/hull $\rightarrow$ local cohomology $\rightarrow$ dualizing/Bass.

This ordering reflects numerical conditioning. Initial shadows and congruence labels are local. Local cohomology and Bass warnings are higher-order support objects and should not be enabled before the representation has stable lower-order structure.

13.5 Minimal open-source reproduction

A minimal reproduction can be done without building all of ToricGT:

1. Train a small graph-token Transformer on graph-labeled semigroup normal-form tasks.
2. Add the lattice-ideal normal-form head and initial-shadow loss.
3. Add Hilbert-basis dictionary pruning.
4. Add a retrieval table keyed by K-signatures and compare against embedding-only retrieval.
5. Evaluate on held-out semigroup families and natural-language explanations of the same tasks.
6. Export an int8 low-rank probe version and report bits-per-byte, exact answer accuracy, and artifact bytes.

This ablation is intentionally narrow. It can validate the central claim—that toric algebra certificates improve reasoning or compression—before adding Scarf complexes, local cohomology, or behavior corridors.

13.6 Reporting checklist

Every experiment should report:

1. base validation bits-per-byte and tagged-slice bits-per-byte;
2. exported artifact bytes, not just parameter count;
3. exact certificate accuracy on a random audit sample;
4. relation residual on held-out binomials;
5. Hilbert-basis coverage and dictionary byte cost;
6. Betti/K retrieval lift;
7. local-cohomology false-positive and false-negative rates;
8. proof or graph exact-match accuracy;
9. behavior calibration when coverage holes are present;
10. three-seed confidence intervals or matched-checkpoint comparisons.

A component should be called successful only when the relevant target metric improves without a statistically meaningful regression in ordinary prose bits-per-byte, unless the experiment is explicitly a research-only graph-reasoning model rather than a compressed language model.

14 Wild-card research directions

14.1 Toric algebra for mixture-of-experts routing

A Soft-MoE router can be treated as a semigroup action on expert usage counts. Let $q_t \in \mathbb{N}^E$ record expert counts along a trajectory and let L_{route} identify count vectors that lead to equivalent outputs. The routing toric ideal $I_{L_{\text{route}}}$ gives binomial constraints on routes. A low-rank router can be regularized by

$$\mathcal{L}_{\text{route-toric}} = \sum_{u-v \in \mathcal{B}_{\text{route}}} |\log p(u \mid h) - \log p(v \mid h)|^2,$$

where $p(u \mid h)$ is the probability of a route count. This can reduce mode collapse by identifying routes that are algebraically equivalent while still allowing specialization when equivalence fails. The export decision is simple: store no extra expert unless its route class improves quality per byte.

14.2 Semigroup-valued curriculum scheduling

Training curricula often use ad hoc difficulty scores. A toric algebra curriculum uses semigroup height, saturation depth, projective dimension, and local-cohomology complexity. Define

$$d_{\text{alg}}(r) = \alpha \omega(q_r) + \beta \text{pd}(I_{L_r}) + h o \#(Q_r^{\text{sat}} \setminus Q_r)_{\leq R} + \eta \sum_i \dim H_{\mathfrak{m}}^i(k[Q_r])_{b_r}.$$

Then sample records with probability proportional to $\exp(-d_{\text{alg}}/T_s)$, annealing T_s . This introduces hard algebraic cases gradually while preserving exact difficulty labels.

14.3 Universal grading discovery for hidden features

Part II emphasizes that a lattice ideal has a universal grading by $\mathbb{Z}^n/L$. For hidden states, one can learn the finest quotient of feature directions under which observed relation residuals are homogeneous. Given relation differences $r_j = u_j - v_j$, learn a projection P such that $Pr_j \approx 0$ for true relations and $Pr_j \not\approx 0$ for false relations:

$$\mathcal{L}_{\text{univ}} = \sum_{j \in \mathcal{R}^+} \|Pr_j\|_2^2 - au \sum_{j \in \mathcal{R}^-} \min(\|Pr_j\|_2^2, m) + \lambda \|PP^T - I\|_F^2.$$

The rows of P become learned grading axes. If they align with GraphCG axes, the model has discovered a quotient structure rather than memorizing individual tasks.

14.4 Canonical-module evidence priors

For normal semigroup rings, the canonical module is controlled by interior degrees. In reasoning, interior degrees correspond to states with enough independent evidence to be away from boundary ambiguity. A canonical-module prior increases probability mass on answer states whose evidence degree lies in a learned interior cone and decreases overconfident answers on boundary states:

$$\mathcal{L}_{\omega} = -\log p(y \mid h) + \lambda \mathbf{1}[b \notin \text{relint}(C)] \max(0, c(h) - c_{\partial})^2.$$

This is a mathematically principled version of “be more cautious near the boundary of available evidence.”

14.5 Multigraded Hilbert-scheme view of representation drift

Across training, the set of hidden relations satisfied by a layer changes. Treat each checkpoint as a point in an approximate multigraded Hilbert scheme of relation ideals. A drift metric can be defined by comparing Hilbert functions of the hidden relation ideal:

$$D_{\text{Hilb}}(\theta_s, \theta_t) = \sum_{a \in A_{\leq R}} |\dim(S/I_{\theta_s})_a - \dim(S/I_{\theta_t})_a|.$$

Large drift late in training signals that the representation is changing its quotient structure and may invalidate cached probes. This gives a principled checkpoint-triage metric.

14.6 Integer-programming certificates as tool-use supervision

The lattice-ideal and Hilbert-basis material is tightly connected to integer programming. For tool-using agents, a tool-call plan can be treated as a feasible integer point. A certificate that a plan is irreducible, saturated, or equivalent to another plan becomes a compact supervision target. Instead of training only on final tool outputs, train on plan normal forms and irreducible decompositions. This can reduce wasted calls and improve compliance because forbidden tool combinations become monomial nonfaces or semigroup holes.

15 Conclusion

Part II of *Combinatorial Commutative Algebra* offers a surprisingly direct training language for ToricGT. Semigroup rings encode reachable reasoning states; lattice ideals encode equivalences of trajectories; Hilbert bases give minimal dictionaries; initial ideals match ReLU/tropical active shadows; K-polynomials and multidegrees give compact signatures; Betti numbers and Scarf/hull complexes describe dependency structure; toric quotients train invariance; injective resolutions and Bass numbers classify failure support; Ehrhart and Brion theory compress state counts; and local cohomology detects holes.

The primary goal remains model improvement. The mathematical objects are not decorations. They are useful only when they reduce held-out bits-per-byte, improve proof and graph reasoning, increase retrieval precision, improve behavior controllability, or provide reliable audits that enable safer training. The most promising immediate experiments are conservative: audit first, train low-rank probes second, enable monomial shadows before binomial lifts, use exact small certificates, cache aggressively, and export nothing unless the bits-per-byte gate or a predeclared reasoning gate passes.

The larger research program is to make hidden reasoning less shapeless. A trajectory should have algebraic coordinates, support, quotients, generators, syzygies, and holes. When those objects are finite and auditable, they can be used to train better reasoning agents without requiring the deployed agent to carry the full machinery of a computer-algebra system.

References

- [1] Ezra Miller and Bernd Sturmfels. *Combinatorial Commutative Algebra*. Graduate Texts in Mathematics 227. Springer, 2005.
- [2] David Cox, John Little, and Hal Schenck. *Toric Varieties*. American Mathematical Society, 2011.
- [3] Bernd Sturmfels. *Groebner Bases and Convex Polytopes*. American Mathematical Society, 1996.
- [4] Dave Bayer and Bernd Sturmfels. Cellular resolutions of monomial modules. *Journal fuer die reine und angewandte Mathematik*, 1998.

- [5] Irena Peeva and Bernd Sturmfels. Generic lattice ideals. *Journal of the American Mathematical Society*, 1998.
- [6] Winfried Bruns and Joseph Gubeladze. *Polytopes, Rings, and K-Theory*. Springer, 2009.
- [7] Amelie Schreiber. *ToricGT with Toric BGG Supervision: Graph-Token Reasoning, Hypertoric Category O, and Bits-per-byte-Constrained Training*. Manuscript, 2026.